Objektorientierte Programmierung

WOCHE 6: FORTGESCHRITTENE KONZEPTE UND PROJEKTSTART

PETER BERNHARDT

März 2026

1

Teil 1: Einführung & Das Warten auf Daten

2

Willkommen zu Woche 6

Wir haben den theoretischen Gipfel dieses Kurses erreicht.

Architekturen, Patterns, SOLID und Testing liegen hinter uns.

Heute betrachten wir die Brücke zur realen Welt:
Wie interagieren unsere sauberen Systeme effizient und ausfallsicher mit der Außenwelt (APIs, Datenbanken)?

Zudem geben wir heute den Startschuss für Ihre praktischen Abschlussprojekte.

3

Die vernetzte Welt

Kein modernes Fachverfahren arbeitet mehr isoliert.

Wir müssen das zentrale Melderegister abfragen, PDF-Generatoren in der Cloud ansprechen oder Geodaten bei einem Karten-Dienst laden.

Diese Abfragen über das Netzwerk oder die Festplatte benötigen Zeit.

Manchmal Millisekunden, manchmal Sekunden.

Manchmal geht es gar nicht…
(Aber darüber denken wir heute mal nicht nach.)

4

Das Problem der Synchronität

Wenn wir traditionell (**synchron**) programmieren, blockiert, also wartet, der aktuelle Thread unseres Programms, bis die externe Antwort da ist.

In einer Desktop-Anwendung führt das dazu, dass die Oberfläche “einfriert” (Keine Rückmeldung).

In einer Web-Anwendung blockieren wir den Webserver für andere Nutzer.

Irgendwie nicht ideal.

5

Die Lösung: Asynchrone Ausführung

Asynchronität heißt:

Das Programm wartet nicht direkt auf die Antwort. Es vergibt eine Aufgabe, macht in der Wartezeit etwas anderes und arbeitet weiter, sobald die Daten angekommen sind.

Analogie:

Asynchronität ist wie eine Bestellung im Selbstbedienungs-Restaurant: Du gibst sie auf, setzt dich wieder an den Tisch und machst etwas anderes. Wenn das Essen fertig ist, wirst du informiert und machst dann weiter (holen, essen, etc.).

6

Teil 2:
Asynchrone Programmierung (async / await)

7

I/O-Bound vs. CPU-Bound

Es gibt zwei Arten von langwierigen Aufgaben in der Programmierung:

1. **I/O-Bound:** Warten auf Netzwerk, Festplatte oder Datenbank. Hier ist *Asynchronität* perfekt, da die CPU während des Wartens nichts zu tun hat.
2. **CPU-Bound:** Komplexe Berechnungen (z.B. Verschlüsselung). Hier nutzt man eher *Parallelität* auf mehreren Kernen, da die CPU aktiv arbeitet.

Für die Parallelität haben wir leider nicht viel Zeit…

8

C# und die Task Parallel Library (TPL)

C# hat asynchrone Programmierung stark vereinfacht.

Anstatt komplexe “Callbacks” zu schreiben, bietet C# zwei magische Schlüsselwörter: `async` und `await`.

Zusammen mit der Klasse `Task<T>` ermöglichen sie uns, asynchronen Code zu schreiben, der sich fast genau so einfach liest wie synchroner Code.

Man hat also die Komplexität der asynchronen Programmierung gekapselt und dabei sehr transparent gemacht.

Und ja, andere Programmiersprachen können das auch :-)

9

Wie funktioniert ein Task?

Ein `Task` repräsentiert eine laufende Operation.

Er ist wie ein “Gutschein” für ein zukünftiges Ergebnis.

- `Task`: Eine Operation ohne Rückgabewert (wie `void`).
- `Task<T>`: Eine Operation, die ein Ergebnis vom Typ `T` liefern wird.

Die Klasse `Task` kümmert sich um die Verwaltung der Threads und die Synchronisation, damit wir uns auf die Logik konzentrieren können.

10

Die Schlüsselworte: **`async`** und **`await`**

Das `async`-Schlüsselwort markiert eine Methode als asynchron. Es ermöglicht die Nutzung von `await` in dieser Methode.

Das `await`-Schlüsselwort bewirkt, dass die Ausführung der Methode pausiert wird, bis der Task abgeschlossen ist.

Währenddessen wird der aufrufende Thread freigegeben und kann andere Aufgaben bearbeiten. Sobald der Task fertig ist, kehrt die Methode an genau diese Stelle zurück.

Der Aufruf einer asynchronen Methode entkoppelt also die Anfrage von der Verarbeitung und meldet sich zurück, sobald die Daten verfügbar sind.

11

Beispiel: Asynchrone Abfrage (Code)

Wir kennzeichnen die Methode mit `async Task<T>` und warten mit `await` auf das Netzwerk.

```
// Die Methode gibt keinen string zurück, sondern ein "Versprechen" (Task)
public async Task<string> HoleBuergerDatenAsync(string id)
{
    HttpClient client = new HttpClient();

    // await "pausiert" diese Methode hier, ohne den Thread zu blockieren!
    string daten = await client.GetStringAsync($"https://api.behoerde.de/{id}");

    // Der Dienst ermittelt nun die Antwort, was u.U. einige Zeit dauern kann...

    // Der Thread wartet nicht untätig, sondern kann in der Zwischenzeit andere
    // Anfragen bearbeiten oder die UI responsive halten.

    // Wenn die Antwort vom Dienst dann da ist, geht es hier weiter
    return daten;
}
```

12

Parallelität: `Task.WhenAll`

Noch mächtiger wird es, wenn wir Daten aus *mehreren* Quellen brauchen.

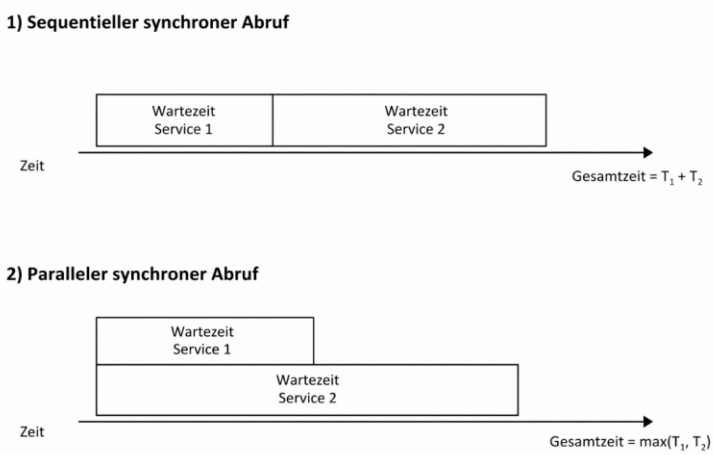
Anstatt Quelle A und danach erst Quelle B abzufragen, können wir beide Anfragen gleichzeitig auf die Reise schicken.

Vorteil: Durch parallele Abfragen kann sich die Gesamtwartezeit drastisch reduzieren. Statt der Summe aller Wartezeiten warten wir nur so lange, wie die langsamste Einzelabfrage dauert.

In vernetzten Systemen kann das ein massiver Performance-Gewinn sein.

13

Laufzeitvergleich ohne/mit Parallelität



14

Beispiel: Parallele API-Abfragen (Code)

```
public async Task<BuergerDaten> HoleVollstaendigeDatenAsync(string id)
{
    // Wir starten zwei Abfragen gleichzeitig (parallel)
    var meldeTask = _apiClient.GetMelderegisterDatenAsync(id);
    var steuerTask = _apiClient.GetSteuerDatenAsync(id);

    // Hier warten wir auf beide Ergebnisse,
    // ohne den Thread zu blockieren
    await Task.WhenAll(meldeTask, steuerTask);

    return new BuergerDaten
    {
        Name = meldeTask.Result.Name,
        SteuerKlasse = steuerTask.Result.Klasse
    };
}
```

15

Die goldene Regel: “Async all the way”

Wenn Sie im "Keller" Ihrer Architektur (Data Access Layer) asynchron werden, sollten Sie die Asynchronität bis ganz nach oben (Presentation Layer) durchziehen.

`async` ist wie ein “Virus”, das sich im positiven Sinne durch den Call-Stack frisst.

Eine synchrone Methode, die eine asynchrone Methode aufruft, ist ein Anti-Pattern.

Denn dann müssen Sie entweder `.Wait()` oder `.Result` verwenden, um das Ergebnis zu bekommen, was die Vorteile der Asynchronität zunichte macht.

Zusätzlich kann in der UI das Programm einfrieren, da der GUI-Thread blockiert wird.

16

Die Deadlock-Gefahr

Nutzen Sie **niemals** `.Wait()` oder `.Result` auf einem `Task` innerhalb einer normalen (synchronen) Methode, um Asynchronität zu erzwingen.

In UI-Frameworks (WPF) oder Web-Kontexten führt dies fast unweigerlich zu **Deadlocks** (das Programm verhakt sich in sich selbst und steht still).

```
public string HoleBuergerDaten(string id)
{
    HttpClient client = new HttpClient();

    // Der Aufruf blockiert Thread bis die Antwort da ist!
    string daten = client.GetStringAsync($"https://api.behoerde.de/{id}").Result;

    return daten;
}
```

Regel: Wo ein `Task` ist, muss ein `await` sein.

17

Teil 3:
Robustes Error Handling

18

Warum `try-catch` allein nicht reicht

Manchmal tut man einfach alles, um Abstürze zu verhindern:

```
try
{
    MacheEtwas();
}
catch // fängt alle möglichen Ausnahmen
{
    // Ignorieren
}
```

In großen Fachverfahren ist dieses Ignorieren (“Verschlucken”) von Fehlern fatal.

Der Bürger denkt z.B., der Antrag ging durch, aber er liegt nie in der Datenbank.

19

Das “Fail Fast” Prinzip

Grundidee: ***Ein System soll so früh wie möglich auf Fehler reagieren und nicht weiterlaufen, wenn etwas offensichtlich schief läuft.***

Das heißt unter anderem, prüfen Sie Eingaben (z.B. Pflichtfelder, leerer Name) und Rahmenbedingungen (z.B. freier Speicher) so früh wie möglich.

Beispiel:
Wenn die `SteuerId` des Antrags leer ist, macht es keinen Sinn, den Service aufzurufen. Werfen Sie sofort eine Exception und brechen Sie ab, bevor teure Ressourcen (z.B. DB-Verbindungen) verbraucht werden.

20

Vorteile von Fail-Fast

1. **Ressourcenschonung:** Keine unnötigen DB-Aufrufe.
2. **Klarheit:** Der Fehler wird dort gemeldet, wo er verursacht wurde.
3. **Sicherheit:** Verhindert, dass das System mit korrupten Daten arbeitet.
4. **Reduzierte Komplexität im Code:** Wenn du früh abbrichst, brauchst du weniger „Was wäre wenn“-Logik.
5. **Bessere Testbarkeit:** Klare Fehlerszenarien lassen sich gezielt testen.

21

Lokales vs. Globales Error Handling

Grundidee: ***Fehler sollten dort gefangen werden, wo sie sinnvoll behandelt werden können.***

1. **Lokal:** Fangen Sie Fehler dort ab, wo Sie sie *behandeln* können (z.B. einen Retry-Mechanismus bei Netzwerkproblemen).
2. **Global:** Nutzen Sie in Anwendungen eine zentrale Middleware, um alle unvorhergesehenen Fehler (im mindesten) zu loggen und dem Nutzer eine freundliche Meldung zu zeigen.

22

Spezifische Exceptions fangen

Grundidee: **Fangen Sie nur Fehler, die Sie auch *sinnvoll behandeln* können.**

Wenn die Festplatte "brennt" (z.B. voll, kaputt), kann Ihr Code das nicht beheben. Lassen Sie das Programm kontrolliert abbrechen!

Fangen Sie spezifische Dinge wie `SqlException`, um z.B. einen Verbindungsversuch 3 Sekunden später nochmal zu wiederholen.
Transiente (also temporäre) Fehler kann man so oft wieder „heilen".

23

Fachliche vs. Technische Fehler

Grundidee: ***Unterscheiden Sie zwischen Fehlern, die durch falsche Benutzereingaben verursacht werden (fachlich) und solchen, die durch Systemfehler entstehen (technisch).***

1. **Fachlich (User-Fehler):** Der Bürger ist zu jung. Zeigen Sie eine nette Meldung: "Sie erfüllen die Voraussetzungen nicht."
2. **Technisch (System-Fehler):** Die Datenbank antwortet nicht. Loggen Sie den Fehler für den Admin und zeigen Sie dem Nutzer: "Ein technisches Problem ist aufgetreten."

24

Fehler-Propagierung

Grundidee: ***Es ist nicht falsch, um Hilfe zu bitten.***

Exceptions sollten nur dann gefangen werden, wenn man sie sinnvoll behandeln kann.

Ansonsten ist es besser, sie nach oben (an den Aufrufer) weiterzureichen, bis sie eine Schicht erreichen, die eine Entscheidung treffen kann (z.B. UI zeigt Fehlermeldung).

Oder sie letztendlich vom globalen Error-Handling zumindest loggen zu lassen.

25

Die Wichtigkeit von Logging

Grundidee: ***Ein Fehler, der nicht geloggt wurde, hat nie stattgefunden.***

Ohne Logdateien sind Sie als Administrator blind.

Wenn um 3 Uhr nachts ein Fehler auftritt, brauchen Sie den Stacktrace, die Uhrzeit und idealerweise die ID des betroffenen Antrags.

Nutzen Sie moderne Bibliotheken wie `Serilog` oder `NLog` und nutzen Sie strukturierte Logs, damit Sie später gezielt nach Fehlern filtern können.

```
logger.Error(ex, "AntragSpeichern: Fehler bei Antrag {antrag.Id}");
```

26

Beispiel: Globales Fehlerhandling (Code)

```
try {
    // Eingaben sollten nicht zu Exceptions führen, sondern vorher validiert werden.
    do {
        bool istFehlerfrei = FrageDenNutzerNachEingaben(out antrag);
    } while (!istFehlerfrei);

    // Hier könnte immer noch ein Fehler passieren (z.B. fehlende externe Daten, DB-Ausfall)
    await _service.VerarbeiteAntragAsync(antrag);

    // Wenn alles gut geht, zeigen wir eine Erfolgsmeldung.
    ZeigeErfolgsMeldung();
}
catch (AntragUngueltigException ex) {
    // Fachlicher Fehler (z.B. Antrag kann nicht bearbeitet werden)
    _logger.Warning("Ungültiger Antrag von User {U}: {Msg}", antrag.UserId, ex.Message);
    ZeigeFehlerMeldung("Bitte prüfen Sie Ihre Eingaben.");
}
catch (Exception ex) {
    // Technischer Fehler (z.B. Datenbank weg)
    _logger.Error(ex, "Kritischer technischer Fehler");
    ZeigeFehlerMeldung("Ein technisches Problem ist aufgetreten. Versuch später erneut.");
}
```

27

Teil 4:
Projektstart & Abschluss

28

Das Abschlussprojekt

Ab heute beginnt die Praxis-Phase.

Sie haben nun den restlichen Kurszeitraum Zeit, ein eigenes Fachverfahren zu entwickeln.

Es geht nicht darum, ein riesiges System mit hunderten Funktionen zu bauen.

Es geht darum, ein **kleines System architektonisch gut** zu konstruieren.

29

Architektur-Anforderungen

Ihr Projekt muss folgende Kriterien erfüllen:

1. **Schichtenarchitektur:** UI, Business Logic und Data Access Layer strikt getrennt.
2. **SOLID:** Bewusste Anwendung der Prinzipien (insb. DI).
3. **Design Patterns:** Mindestens zwei Muster implementiert.
4. **Testing:** Kernlogik mit Unit Tests (xUnit) abgesichert.

30

Clean Code Kriterien

- **Sprechende Namen:** Klassen und Methoden müssen selbsterklärend sein.
- **Kleine Methoden:** Jede Methode sollte nur eine Sache tun.
- **Kommentare:** Nur dort, wo das “Warum” nicht aus dem Code hervorgeht.

31

Dokumentation des Projekts

Schreiben Sie nicht nur Code.

Ein kurzes README, das Ihre Design-Entscheidungen (Welche Patterns? Warum diese Architektur? etc.) erläutert, ist Teil der Abgabe und hilft bei der Bewertung.

Es hilft Ihnen ebenso bei der späteren Präsentation des Projektes vor der Gruppe.

32

Themenvorschläge

- Wohngeld-Prüfmodul
- Kfz-Bestandsverwaltung
- Kita-Priorisierung
- Bürgeramt-Terminreservierung
- Fundbüro-Bestandsführung
- Bauantrags-Statusverfolgung
- Hundesteuer-Veranlagung
- Mängelmelder-System
- Gewerberegister-Eintragung
- Friedhofs-Grabverwaltung

33

Der Workflow für die nächsten beiden Wochen

1. Überlegen Sie sich Ihr Thema und definieren Sie das Kern-Feature.
2. Skizzieren Sie die Klassenstruktur (Welche Schichten? Welche Patterns?).
3. Starten Sie mit der Fachlogik und schreiben Sie Tests (TDD kann helfen).
4. Bauen Sie die UI und die Datenbank-Anbindung erst ganz am Schluss um Ihre Logik herum.

Hinweis: Sie können statt einer echten Datenbank eine InMemory-Datenbank (siehe Übung) verwenden.

34

Zusammenfassung & Startschuss

- Asynchronität und Fehlerbehandlung sind die Kür für den Produktivbetrieb.
- Jetzt liegt es an Ihnen, das Theoriewissen in praktischen, langlebigen Code zu gießen.
- Viel Erfolg bei der Konzeption und Programmierung Ihrer Abschlussprojekte!

Die “Projektwerkstatt” ist hiermit eröffnet. :-)

35